

Codice Server

```
#include <sys/socket.h>
#include <sys/un.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>

#define SOCKET_PATH "/tmp/time_socket"

void gestisci(int sd);

int main(void) {
    int listen_sd, connect_sd;
    struct sockaddr_un my_addr, client_addr;
    socklen_t client_len;

    // Crea il socket locale
    if ((listen_sd = socket(AF_LOCAL, SOCK_STREAM, 0)) < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    // Imposta l'indirizzo del server
    memset(&my_addr, 0, sizeof(my_addr));
    my_addr.sun_family = AF_LOCAL;
    strncpy(my_addr.sun_path, SOCKET_PATH, sizeof(my_addr.sun_path) - 1);
```

Codice Server

```
// Imposta l'indirizzo del server
memset(&my_addr, 0, sizeof(my_addr));
my_addr.sun_family = AF_LOCAL;
strncpy(my_addr.sun_path, SOCKET_PATH, sizeof(my_addr.sun_path) - 1);

// Rimuove eventuale socket precedente
unlink(SOCKET_PATH);

// Effettua il bind
if (bind(listen_sd, (struct sockaddr*)&my_addr, sizeof(my_addr)) < 0)
{
    perror("bind");
    close(listen_sd);
    exit(EXIT_FAILURE);
}

// Mette il socket in ascolto
if (listen(listen_sd, 5) < 0) {
    perror("listen");
    close(listen_sd);
    exit(EXIT_FAILURE);
}
```

Codice Server

```
// Ciclo principale: accetta connessioni e serve i client
while (1) {
    client_len = sizeof(client_addr);
    connect_sd = accept(listen_sd, (struct sockaddr*)&client_addr, &client_len);
    if (connect_sd < 0) {
        perror("accept");
        continue;
    }

    printf("Nuova connessione accettata.\n");
    gestisci(connect_sd);
    close(connect_sd);
}

close(listen_sd);
unlink(SOCKET_PATH);
return 0;
}
```

Codice Server

```
void gestisci(int sd) {
    char buffer[26];
    time_t ora;

    time(&ora);
    ctime_r(&ora, buffer); // converte in stringa leggibile

    printf("Invio ora al client: %s", buffer);
    write(sd, buffer, strlen(buffer));
}
```

Codice Client

```
#include <sys/socket.h>
#include <sys/un.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define SOCKET_PATH "/tmp/time_socket"

int main(void) {
    int sd;
    struct sockaddr_un srv_addr;
    char msg[100];

    // Crea socket
    if ((sd = socket(AF_LOCAL, SOCK_STREAM, 0)) < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    // Imposta indirizzo del server
    memset(&srv_addr, 0, sizeof(srv_addr));
    srv_addr.sun_family = AF_LOCAL;
    strncpy(srv_addr.sun_path, SOCKET_PATH, sizeof(srv_addr.sun_path) - 1);
```

Codice Client

```
// Connessione al server
if (connect(sd, (struct sockaddr*)&srv_addr, sizeof(srv_addr)) < 0) {
    perror("connect");
    close(sd);
    exit(EXIT_FAILURE);
}

// Riceve e stampa l'ora
ssize_t n = read(sd, msg, sizeof(msg) - 1);
if (n > 0) {
    msg[n] = '\0';
    printf("Ora ricevuta dal server: %s", msg);
} else {
    perror("read");
}

close(sd);
return 0;
}
```

Scambio dati: encoding

- Per scambiarsi i dati client e server devono fare encoding-decoding
 - Esempio conversione array-stringa

```
int a[] = {10, 20, 30, 40, 50};
int n = 5;
char msg[100] = "";
char temp[10];          // dichiarata fuori dal ciclo

for (int i = 0; i < n; i++) {
    sprintf(temp, "%d ", a[i]); // converte numero in testo
    strcat(msg, temp);          // aggiunge al messaggio finale
}

write(sd, msg, strlen(msg));    // invia "10 20 30 40 50 "
```

Scambio dati: decoding

- Per scambiarsi i dati client e server devono fare encoding-decoding
 - Esempio: parsing e conversione stringa-array

```
char msg[100];
int vals[10];
int i = 0;

read(sd, msg, sizeof(msg) - 1);
msg[strcspn(msg, "\n")] = '\0'; // trova primo newline e chiude la stringa

char *token = strtok(msg, " "); // divide la stringa in token delimitati da spazi
while (token != NULL) {
    vals[i++] = atoi(token); // converte token in intero
    token = strtok(NULL, " "); // passa al prossimo token
}
```


Scambio dati: binari

- Nel C, un socket trasmette byte, non tipi. Quindi dobbiamo definire noi come rappresentare l'array in byte.
- Problemi da gestire:
 - **Tipi a dimensione fissa:** usare `uint32_t` o `int32_t` da `<stdint.h>`
 - **Ordine dei byte:** usare *network byte order* (`htonl()` / `ntohl()`)
 - **Framing:** indicare quanti elementi si mandano (così il ricevente sa dove finisce)

Scambio dati: binari

- Nel C, un socket trasmette byte, non tipi. Quindi dobbiamo definire noi come rappresentare l'array in byte.
- Client invia n dati di un array

```
uint32_t vals[] = {10, 20, 30, 40, 50};
uint32_t n = sizeof(vals) / sizeof(vals[0]);

// --- Invio dimensione ---
uint32_t n_net = htonl(n);
write(sd, &n_net, sizeof(n_net));

// --- Invio valori convertiti ---
for (uint32_t i = 0; i < n; i++) {
    uint32_t v_net = htonl(vals[i]);
    write(sd, &v_net, sizeof(v_net));
}
```

Scambio dati: binari

- Nel C, un socket trasmette byte, non tipi. Quindi dobbiamo definire noi come rappresentare l'array in byte.
- Server legge n dati di un array

```
// --- Legge prima la dimensione ---
uint32_t n_net, n;
read(sd, &n_net, sizeof(n_net));
n = ntohl(n_net); // converte in ordine host

printf("Server: mi aspetto %u numeri\n", n);
uint32_t *vals = malloc(n * sizeof(uint32_t));

// --- Legge gli elementi uno a uno ---
for (uint32_t i = 0; i < n; i++) {
    uint32_t v_net;
    read(sd, &v_net, sizeof(v_net));
    vals[i] = ntohl(v_net);
}
```

Socket Locali: datagram

- Comunicazione tra processi sulla *stessa macchina*, **senza connessione**. Ogni messaggio è *un datagram complete*
- Caratteristiche:
 - Server legge n dati di un array AF_LOCAL + SOCK_DGRAM
 - Nessun listen() o accept()
 - Si usano sendto() / recvfrom()
 - Ogni messaggio è **indipendente** e **auto-contenuto**

Socket Locali: datagram

- Comunicazione tra processi sulla *stessa macchina*, **senza connessione**. Ogni messaggio è *un datagram complete*
- Server:

```
// Server
int s = socket(AF_LOCAL, SOCK_DGRAM, 0);
struct sockaddr_un addr = { .sun_family = AF_LOCAL };
strcpy(addr.sun_path, "/tmp/sock");
unlink(addr.sun_path);
bind(s, (struct sockaddr*)&addr, sizeof(addr));

char buf[100];
struct sockaddr_un src;
socklen_t slen = sizeof(src);
recvfrom(s, buf, sizeof(buf), 0, (struct sockaddr*)&src, &slen);
printf("Server ha ricevuto: %s\n", buf);
```

Socket Locali: datagram

- Comunicazione tra processi sulla *stessa macchina*, **senza connessione**. Ogni messaggio è *un datagram complete*
- Client:

```
// Client
int c = socket(AF_LOCAL, SOCK_DGRAM, 0);
struct sockaddr_un srv = { .sun_family = AF_LOCAL };
strcpy(srv.sun_path, "/tmp/sock");

sendto(c, "Ciao!", 5, 0, (struct sockaddr*)&srv, sizeof(srv));
```

Socket Locali: datagram

- Comunicazione tra processi sulla *stessa macchina*, **senza connessione**. Ogni messaggio è *un datagram completo*
- Caratteristiche:
 - Ogni `sendto()` → un datagram intero.
 - Non `connect()` né `accept()`.
 - Kernel gestisce la **consegna** e la **dimensione massima** (~64 KB)
 - Se il buffer ricezione è piccolo → il messaggio viene **troncato**.
 - Perfetto per **messaggi brevi e autonomi** (eventi, notifiche, comandi).